

SystemVerilog 从零到数字逻辑实验考试讲义

这份讲义的目标不是把 SystemVerilog 所有语法都讲完，而是让你能独立完成本文件夹里这些数字逻辑实验和历年考核题难度的题目。

从你已有的实验报告和历年题来看，考试核心反复出现这些能力：

1. 会写 `module`、端口、位宽、数制、赋值。
2. 会写组合逻辑：译码器、比较器、选择器、加法器、计数结果判断。
3. 会写时序逻辑：寄存器、复位、计数器、分频、边沿检测、锁存。
4. 会写有限状态机：输入进度、按键次数、模式切换、锁定/解锁。
5. 会把题目文字翻译成硬件结构：输入是什么，内部要存什么，什么时候更新，输出怎么得到。
6. 会写基本 testbench 验证自己的设计。

SystemVerilog 和普通编程最大的区别是：你不是在写一段顺序执行的程序，而是在描述一堆硬件电路。

`assign`、`always_comb`、`always_ff` 都不是“运行到这里再运行下一行”的脚本，它们描述的是并行存在的电路。

0. 学 SV 前必须先换一个脑子

0.1 软件思维和硬件思维

软件代码常常是：

```
a = b + c;  
d = a + 1;
```

这表示 CPU 先执行第一句，再执行第二句。

硬件描述更像是在说：

```
assign a = b + c;  
assign d = a + 1;
```

这表示同时存在两块组合电路：

1. 一块加法器算出 `a`。
2. 另一块加法器根据 `a` 算出 `d`。

只要 `b` 或 `c` 变化，`a` 和 `d` 就会跟着变化。

0.2 两类电路

数字电路大体分两类。

组合逻辑

输出只由当前输入决定，没有记忆。

例子：

```
assign y = (a & b) | c;
```

译码器、加法器、比较器、多路选择器都是组合逻辑。

时序逻辑

输出和内部状态有关，需要时钟保存历史。

例子：

```
always_ff @(posedge clk or posedge rst) begin
    if (rst)
        count <= 4'd0;
    else
        count <= count + 4'd1;
end
```

计数器、秒表、密码锁、老虎机锁定状态、按键次数统计都是时序逻辑。

考试题基本都可以拆成：

```
输入开关/按键/时钟
-> 时序部分保存状态、计数、分频、锁定
-> 组合部分根据状态计算显示和判断
-> LED/数码管输出
```

1. 最小 SV 程序：module

1.1 module 是硬件模块

一个 .sv 文件里可以有一个或多个 module。每个 module 描述一个硬件模块。

```
module and_gate (
    input  logic a,
    input  logic b,
    output logic y
);

    assign y = a & b;
```

```
endmodule
```

端口方向：

```
input    // 输入
output   // 输出
inout    // 双向，实验里基本不用
```

常用写法：

```
input logic      clk,
input logic      rst,
input logic [3:0] sw,
output logic [6:0] seg
```

`logic [3:0] sw` 表示 4 位信号，位号是：

```
sw[3] sw[2] sw[1] sw[0]
```

通常 `sw[3]` 是高位，`sw[0]` 是低位。

1.2 实验中最常见的顶层模块

```
module top (
    input logic      CLK,
    input logic      RST,
    input logic [3:0] Code,
    output logic [6:0] seg,
    output logic      led
);

    // 内部信号写这里

endmodule
```

注意：端口名必须和约束文件 `.xdc` 的 `get_ports` 对得上。

2. wire、logic、reg 到底是什么

2.1 推荐规则

在 SystemVerilog 里，你可以先记这个考试够用的规则：

1. 大部分信号用 `logic`。
2. 用 `assign` 连出来的内部线，也可以用 `logic`。
3. 如果老师或旧代码写 `wire`，理解为“线网”。
4. 如果旧代码写 `reg`，在 SV 里基本可以换成 `logic`。
5. 一个信号只能有一个主要驱动源，不要又 `assign` 又在 `always_ff` 里赋值。

2.2 wire

`wire` 表示线网，适合连续赋值或模块连接。

```
wire a;  
wire [3:0] sum;  
  
assign a = b & c;
```

早期 Verilog 里，`assign` 左边常写 `wire`。

2.3 reg

`reg` 是老 Verilog 里的变量类型。名字很误导，`reg` 不一定综合成寄存器。

```
reg [3:0] x;  
  
always_comb begin  
    x = a + b;    // 这是组合逻辑，不一定是寄存器  
end
```

是否变成寄存器，看它是不是在时钟边沿里赋值。

2.4 logic

`logic` 是 SystemVerilog 推荐类型，可以替代大多数 `wire/reg` 用法。

```
logic [3:0] count;  
logic      tick;
```

实践规则：

```
// assign 驱动  
logic y;  
assign y = a & b;  
  
// always_comb 驱动  
logic [3:0] out;  
always_comb begin
```

```
        out = in + 4'd1;
    end

    // always_ff 驱动
    logic [3:0] count;
    always_ff @(posedge clk or posedge rst) begin
        if (rst)
            count <= 4'd0;
        else
            count <= count + 4'd1;
    end
```

3. 数值、位宽、数制

3.1 标准写法

SV 数字常写成：

位宽'进制数值

例子：

```
4'b1010    // 4 位二进制，等于十进制 10
4'd10      // 4 位十进制 10
4'hA       // 4 位十六进制 A，也等于 10
8'hFF      // 8 位十六进制 FF，等于 255
16'h1234   // 16 位十六进制数，常用于四位密码
20'd999999 // 20 位十进制数，用于 1 MHz 分频到 1 秒
```

进制字母：

b 二进制
d 十进制
h 十六进制
o 八进制，少用

3.2 为什么一定要写位宽

不推荐：

```
assign x = 10;
```

推荐：

```
assign x = 4'd10;
```

原因是硬件很在乎位宽。比如：

```
logic [3:0] a;  
logic [3:0] b;  
logic [3:0] sum4;  
logic [4:0] sum5;  
  
assign sum4 = a + b; // 最高进位会被截掉  
assign sum5 = a + b; // 可以保存 0~30
```

如果 $a=15$, $b=15$ ：

```
真实结果 30 = 5'b11110  
sum4 只剩低 4 位 1110 = 14  
sum5 是 11110 = 30
```

四位加法器如果要输出进位，就要用 5 位保存结果：

```
logic [4:0] sum;  
assign sum = A + B + Cin;  
assign F = sum[3:0];  
assign Cout = sum[4];
```

3.3 下划线可以让数字更好读

```
20'd999_999  
32'd100_000_000
```

下划线不影响数值。

3.4 拼接和切片

拼接：

```
logic [15:0] token;  
logic [3:0] digit;  
logic [15:0] full_code;  
  
assign full_code = {token[15:4], digit};
```

`{a, b}` 表示把 `a` 放高位, `b` 放低位。

切片:

```
random_out[7:6] // 取高 2 位  
random_out[5:3] // 取中间 3 位  
random_out[2:0] // 取低 3 位
```

考试题经常让你把 LFSR 随机数分段送给数码管:

```
assign dice1 = {2'b00, random_out[7:6]};  
assign dice2 = {1'b0, random_out[5:3]};  
assign dice3 = {1'b0, random_out[2:0]};
```

4. 运算符

4.1 逻辑/位运算

```
&    // 按位与  
|    // 按位或  
^    // 按位异或  
~    // 按位取反  
!    // 逻辑非  
&&  // 逻辑与  
||   // 逻辑或
```

例子:

```
assign y = (a & b) | c;  
assign same = (A == B);  
assign hit = mole & hammer;
```

如果 `mole` 和 `hammer` 都是 4 位:

```

logic [3:0] hit_each;
logic      any_hit;
logic      hit_empty;

assign hit_each  = mole & hammer;      // 每个洞是否击中
assign any_hit   = |hit_each;          // 归约或：只要有一个 1 就是 1
assign hit_empty = |(hammer & ~mole);  // 锤子打到没有地鼠的位置

```

|hit_each 叫归约或，把多位压成一位。

4.2 比较运算

```

==  !=
>   >=
<   <=

```

例子：

```

assign dense = (ones_count >= 4'd4);
assign pass  = (sum >= 4'd10);

```

在 testbench 里常用 != 检查 X/Z：

```

if (out != expected) begin
    $display("error");
end

```

4.3 算术运算

```

+   -   *   /   %

```

考试和实验里 +、- 很常用。

/ 和 % 可以用于小范围常量除法，比如把 0~31 拆成十位个位：

```

assign Tens = sum / 10;
assign Ones = sum % 10;

```

但硬件上除法比较重。考试题如果只是 0~9 或 0~15，优先用比较、case 或简单映射。

4.4 三目运算符

```
assign y = sel ? a : b;
```

意思：

```
sel 为 1, y=a  
sel 为 0, y=b
```

例子：

```
assign display_value = pause ? saved_value : current_value;
```

5. 三种核心写法：assign、always_comb、always_ff

5.1 assign：连续赋值

适合简单组合逻辑。

```
assign sum = A + B;  
assign led = (count >= 4'd10);  
assign gear = speed / 2 + 1;
```

assign 描述的是一根线接到一块组合电路上。右边输入一变，左边立刻跟着变。

5.2 always_comb：复杂组合逻辑

适合 **case**、多层 **if**、译码器。

七段译码器：

```
module decoder (  
    input  logic [3:0] sw,  
    output logic [6:0] seg  
);  
  
    always_comb begin  
        case (sw)  
            4'h0: seg = 7'b1111110;  
            4'h1: seg = 7'b0110000;  
            4'h2: seg = 7'b1101101;  
            4'h3: seg = 7'b1111001;  
            4'h4: seg = 7'b0110011;  
        endcase  
    end
```

```
        4'h5: seg = 7'b1011011;
        4'h6: seg = 7'b1011111;
        4'h7: seg = 7'b1110000;
        4'h8: seg = 7'b1111111;
        4'h9: seg = 7'b1110011;
        4'hA: seg = 7'b1110111;
        4'hB: seg = 7'b0011111;
        4'hC: seg = 7'b1001110;
        4'hD: seg = 7'b0111101;
        4'hE: seg = 7'b1001111;
        4'hF: seg = 7'b1000111;
        default: seg = 7'b0000000;
    endcase
end
endmodule
```

组合逻辑必须保证每条路径都赋值，否则会意外综合出锁存器。

危险写法：

```
always_comb begin
    if (sel)
        y = a;
    // sel=0 时 y 没有赋值，会产生 latch
end
```

正确写法：

```
always_comb begin
    if (sel)
        y = a;
    else
        y = b;
end
```

或者先给默认值：

```
always_comb begin
    y = b;
    if (sel)
        y = a;
end
```

5.3 always_ff：时钟触发的寄存器

只要需要“记住以前的值”，就要用 `always_ff`。

```
always_ff @(posedge clk or posedge rst) begin
    if (rst)
        q <= 1'b0;
    else
        q <= d;
end
```

这就是一个 D 触发器：

复位时 `q=0`
否则每个 `clk` 上升沿，把 `d` 保存进 `q`

常见模板：

```
always_ff @(posedge CLK or posedge RST) begin
    if (RST) begin
        // 复位值
    end else begin
        // 正常时钟更新
    end
end
```

考试里基本都用高电平异步复位：

```
posedge RST
```

如果题目或模板说低电平复位，才写：

```
negedge rst_n
```

6. 阻塞赋值 = 和非阻塞赋值 <=

这是初学者最容易混的地方。

6.1 组合逻辑用 =

```
always_comb begin
    y = a;
```

```
    z = y;  
end
```

在这个块里，`z` 会得到新的 `y`。

6.2 时序逻辑用 `<=`

```
always_ff @(posedge clk) begin  
    a <= b;  
    b <= a;  
end
```

这表示在同一个时钟沿，`a` 得到旧的 `b`，`b` 得到旧的 `a`，两者交换。

不要在 `always_ff` 里大量使用 `=`。考试记住：

```
always_comb 用 =  
always_ff   用 <=  
assign      用 =
```

6.3 非阻塞赋值的关键现象

看这个秒表分频代码：

```
always_ff @(posedge CLK or posedge RST) begin  
    if (RST) begin  
        div_cnt <= 20'd0;  
        tick_1s <= 1'b0;  
    end else begin  
        if (div_cnt == 20'd999_999) begin  
            div_cnt <= 20'd0;  
            tick_1s <= 1'b1;  
        end else begin  
            div_cnt <= div_cnt + 20'd1;  
            tick_1s <= 1'b0;  
        end  
    end  
end  
end
```

`tick_1s <= 1'b1` 会在这个时钟沿结束后生效。如果另一个 `always_ff` 也在同一个 `posedge CLK` 里读取 `tick_1s`，读到的是旧值。

所以更稳的写法是把“达到终值”做成组合信号：

```
logic tick_now;
assign tick_now = (div_cnt == 20'd999_999);

always_ff @(posedge CLK or posedge RST) begin
    if (RST)
        div_cnt <= 20'd0;
    else if (tick_now)
        div_cnt <= 20'd0;
    else
        div_cnt <= div_cnt + 20'd1;
end

always_ff @(posedge CLK or posedge RST) begin
    if (RST)
        sec <= 6'd0;
    else if (tick_now)
        sec <= (sec == 6'd59) ? 6'd0 : sec + 6'd1;
end
```

这样两个块都看同一个条件，时序更清楚。

7. 组合逻辑基本功

7.1 译码器

译码器本质是查表。

BCD/十六进制到七段码：

```
always_comb begin
    case (value)
        4'd0: seg = 7'b1111110;
        4'd1: seg = 7'b0110000;
        4'd2: seg = 7'b1101101;
        4'd3: seg = 7'b1111001;
        4'd4: seg = 7'b0110011;
        4'd5: seg = 7'b1011011;
        4'd6: seg = 7'b1011111;
        4'd7: seg = 7'b1110000;
        4'd8: seg = 7'b1111111;
        4'd9: seg = 7'b1110011;
        default: seg = 7'b0000000;
    endcase
end
```

如果题目说“带译码数码管”，通常你的输出不是七段 `seg[6:0]`，而是 4 位 BCD 值：

```
output logic [3:0] Speed,  
output logic [3:0] Gear
```

这时你只要输出数字 0~9:

```
assign Speed = speed;  
assign Gear = gear;
```

如果题目说“七段数码管段选”，才需要输出 `seg[6:0]`。

7.2 多路选择器

二选一:

```
assign y = sel ? a : b;
```

多选一:

```
always_comb begin  
    case (sel)  
        2'd0: y = a;  
        2'd1: y = b;  
        2'd2: y = c;  
        default: y = d;  
    endcase  
end
```

考试例子：暂停状态下切换查看当前随机序列和保存序列。

```
assign shown_seq = show_saved ? saved_seq : current_seq;
```

7.3 加法器

四位加法器直接写:

```
module adder (  
    input logic [3:0] A,  
    input logic [3:0] B,  
    input logic Cin,  
    output logic [3:0] F,  
    output logic Cout
```

```
);

    logic [4:0] sum;

    assign sum = A + B + Cin;
    assign F = sum[3:0];
    assign Cout = sum[4];

endmodule
```

如果题目要求“用全加器串联”，写子模块：

```
module full_adder (
    input  logic A,
    input  logic B,
    input  logic Cin,
    output logic S,
    output logic Cout
);

    assign S = A ^ B ^ Cin;
    assign Cout = (A & B) | (Cin & (A ^ B));

endmodule
```

实例化：

```
full_adder u0 (
    .A(A[0]),
    .B(B[0]),
    .Cin(Cin),
    .S(F[0]),
    .Cout(c0)
);
```

端口连接推荐永远用 **.端口名(信号名)**，不要用位置连接。

7.4 比较和范围判断

汽车档位题：

```
速度 0/1 -> 档位 1
速度 2/3 -> 档位 2
速度 4/5 -> 档位 3
速度 6/7 -> 档位 4
速度 8/9 -> 档位 5
```

可以写：

```
always_comb begin
    if (speed <= 4'd1)
        gear = 4'd1;
    else if (speed <= 4'd3)
        gear = 4'd2;
    else if (speed <= 4'd5)
        gear = 4'd3;
    else if (speed <= 4'd7)
        gear = 4'd4;
    else
        gear = 4'd5;
end
```

也可以利用规律：

```
assign gear = (speed >> 1) + 4'd1;
```

但要注意 speed 只能是 0~9。

7.5 统计 1 的个数

2025 序列检测器题要求判断 8 位随机序列中 1 的数量是否大于等于 4。

写法：

```
logic [3:0] ones_count;

assign ones_count = seq[0] + seq[1] + seq[2] + seq[3] +
                    seq[4] + seq[5] + seq[6] + seq[7];

assign dense = (ones_count >= 4'd4);
```

注意 `seq[i]` 是 1 位，加起来时最好接到 4 位结果。

7.6 打地鼠的命中判断

```
logic [3:0] mole;        // 1 表示有地鼠
logic [3:0] hammer;      // 1 表示打这个洞
logic [3:0] hit_each;
logic      any_hit;
logic      hit_empty;
logic      success_req2;
logic      success_req3;
```



```
assign hit_each = mole & hammer;
assign any_hit = |hit_each;
assign hit_empty = |(hammer & ~mole);

// 要求2: 至少打中一个, 且没打空
assign success_req2 = any_hit && !hit_empty;

// 要求3: 出现多个也必须全打中, 且没打空, 且按了确认键
assign success_req3 = (hammer == mole) && (mole != 4'b0000) && confirm_pressed;
```

这类题的关键不是语法, 而是把自然语言翻译成布尔表达式。

8. 时序逻辑基本功

8.1 寄存器: 保存一个值

```
logic [7:0] saved_seq;

always_ff @(posedge CLK or posedge RST) begin
    if (RST)
        saved_seq <= 8'd0;
    else if (need_save)
        saved_seq <= current_seq;
end
```

题目说“保存上一次”“锁定当前”“停止刷新”“记住密码”, 都需要寄存器。

8.2 计数器

普通递增:

```
always_ff @(posedge CLK or posedge RST) begin
    if (RST)
        count <= 4'd0;
    else
        count <= count + 4'd1;
end
```

带上限循环:

```
always_ff @(posedge CLK or posedge RST) begin
    if (RST)
        count <= 4'd0;
    else if (count == 4'd9)
        count <= 4'd0;
    else
```

```
        count <= count + 4'd1;
    end
```

带使能：

```
always_ff @(posedge CLK or posedge RST) begin
    if (RST)
        count <= 4'd0;
    else if (enable) begin
        if (count == 4'd9)
            count <= 4'd0;
        else
            count <= count + 4'd1;
    end
end
```

8.3 1 MHz 分频到 1 秒

题目经常给：

时钟输入频率为 1 MHz
每 1 秒更新一次

1 MHz 表示每秒 1,000,000 个时钟周期。所以计数 0 到 999,999。

```
logic [19:0] div_cnt;
logic        tick_1s;

always_ff @(posedge CLK or posedge RST) begin
    if (RST) begin
        div_cnt <= 20'd0;
        tick_1s <= 1'b0;
    end else begin
        if (div_cnt == 20'd999_999) begin
            div_cnt <= 20'd0;
            tick_1s <= 1'b1;
        end else begin
            div_cnt <= div_cnt + 20'd1;
            tick_1s <= 1'b0;
        end
    end
end
```

如果是 2 秒：

```
if (div_cnt == 21'd1_999_999)
```

如果是 3 秒：

```
if (div_cnt == 22'd2_999_999)
```

如果是 0.2 秒：

```
if (div_cnt == 18'd199_999)
```

位宽怎么估：

```
2^18 = 262144, 可放 199999
2^20 = 1048576, 可放 999999
2^21 = 2097152, 可放 1999999
2^22 = 4194304, 可放 2999999
```

懒一点可以用 32 位：

```
logic [31:0] div_cnt;
```

考试可读性更重要，32 位一般没问题。

8.4 秒表 00 到 59

```
logic [3:0] tens;
logic [3:0] ones;

always_ff @(posedge CLK or posedge RST) begin
    if (RST) begin
        tens <= 4'd0;
        ones <= 4'd0;
    end else if (tick_1s) begin
        if (ones == 4'd9) begin
            ones <= 4'd0;
            if (tens == 4'd5)
                tens <= 4'd0;
            else
                tens <= tens + 4'd1;
        end else begin
            ones <= ones + 4'd1;
        end
    end
end
```

```
end
end
```

这个模板可迁移到很多题：

1. 个位满 9 向十位进位。
2. 到 59 回到 00。
3. 有 `tick_1s` 才更新。

8.5 边沿检测

题目常说：

每当按键按下并释放后，上升沿触发
检测到上升沿后执行一次

边沿检测需要保存上一个周期的按键值。

```
logic key_d;
logic key_rise;

always_ff @(posedge CLK or posedge RST) begin
    if (RST)
        key_d <= 1'b0;
    else
        key_d <= key;
end

assign key_rise = key & ~key_d;
```

如果你的按键是手动时钟本身，比如 `CLK` 就来自按键，那不需要再检测这个按键的边沿，`posedge CLK` 已经是按下一次触发一次。

如果题目给 1 MHz 主时钟，同时另有微动开关，那就要用上面这种同步边沿检测。

实际板子上的机械按键会抖动，严格工程要消抖。实验考试一般不要求，题面说“按键上升沿”时通常写边沿检测即可。

8.6 LFSR 随机数

历年题常给这段：

```
module LFSR_random_generator (
    input logic    clk,
    input logic    reset,
    output logic [7:0] random_out
);
```

```
logic [7:0] lfsr;

always_ff @(posedge clk or posedge reset) begin
    if (reset)
        lfsr <= 8'b0000_0001;
    else
        lfsr <= {lfsr[6:0], lfsr[7] ^ lfsr[5] ^ lfsr[4] ^ lfsr[3]};
end

assign random_out = lfsr;

endmodule
```

你要会做两件事：

1. 实例化 LFSR。
2. 决定什么时候更新显示。

不要把 LFSR 的 `clk` 接到 1 MHz 后又直接把 `random_out` 送显示，否则它会每微秒乱跳。考试通常要求 1 秒、2 秒、3 秒、0.2 秒刷新，所以要用寄存器锁存显示值：

```
logic [7:0] random_out;
logic [7:0] shown_random;

LFSR_random_generator u_lfsr (
    .clk(CLK),
    .reset(RST),
    .random_out(random_out)
);

always_ff @(posedge CLK or posedge RST) begin
    if (RST)
        shown_random <= 8'd0;
    else if (tick_1s)
        shown_random <= random_out;
end
```

输出用 `shown_random`，不要直接用 `random_out`。

9. 有限状态机 FSM

9.1 什么时候需要状态机

题目里出现这些词，基本就是 FSM：

第 1 次按键做 A，第 2 次做 B，第 3 次做 C
输入第 1 位、第 2 位、第 3 位、第 4 位

运行、暂停、查看保存值
 锁定、解锁、报警
 模式切换

9.2 状态机三要素

1. 当前状态 `state`
2. 下一状态 `next_state`
3. 输出和寄存器更新

小题可以把状态转移和动作写在一个 `always_ff` 里。复杂题更推荐分开。

9.3 enum 定义状态

```
typedef enum logic [1:0] {
    ROLL_ALL = 2'd0,
    LOCK_D1  = 2'd1,
    LOCK_D2  = 2'd2,
    LOCK_D3  = 2'd3
} state_t;

state_t state;
```

比直接写 `logic [1:0] state` 清楚很多。

9.4 分步骰子题状态机

题意：

第1次按键：锁定骰子1，骰子2/3继续刷新
 第2次按键：锁定骰子2，骰子3继续刷新
 第3次按键：锁定骰子3，计算总和
 复位：全部恢复刷新，LED 熄灭

核心代码：

```
typedef enum logic [1:0] {
    ROLL_ALL = 2'd0,
    LOCK_1   = 2'd1,
    LOCK_2   = 2'd2,
    LOCK_3   = 2'd3
} state_t;

state_t state;

logic [3:0] d1, d2, d3;
logic [3:0] r1, r2, r3;
```

```

logic      led;

always_ff @(posedge CLK or posedge RST) begin
    if (RST) begin
        state <= ROLL_ALL;
        d1 <= 4'd0;
        d2 <= 4'd0;
        d3 <= 4'd0;
        led <= 1'b0;
    end else begin
        if (tick_1s) begin
            if (state == ROLL_ALL)
                d1 <= r1;
            if (state == ROLL_ALL || state == LOCK_1)
                d2 <= r2;
            if (state == ROLL_ALL || state == LOCK_1 || state == LOCK_2)
                d3 <= r3;
        end

        if (key_rise) begin
            case (state)
                ROLL_ALL: begin
                    d1 <= r1;
                    state <= LOCK_1;
                end
                LOCK_1: begin
                    d2 <= r2;
                    state <= LOCK_2;
                end
                LOCK_2: begin
                    d3 <= r3;
                    led <= ((d1 + d2 + r3) >= 4'd10);
                    state <= LOCK_3;
                end
                default: begin
                    state <= LOCK_3;
                end
            endcase
        end
    end
end
end

```

注意第三次按键时 `d3 <= r3` 和 `led <= ...` 同时发生，`led` 不能用新 `d3`，要用 `r3`。

这就是非阻塞赋值的重要性。

9.5 密码锁输入状态机

四位密码输入：

S0 等第 1 位
S1 等第 2 位

S2 等第 3 位
S3 等第 4 位并判断
S4 保持结果，等待复位开始下一轮

框架：

```
typedef enum logic [2:0] {
    S0 = 3'd0,
    S1 = 3'd1,
    S2 = 3'd2,
    S3 = 3'd3,
    S4 = 3'd4
} state_t;

state_t state;
logic [15:0] token;
logic [15:0] password;
logic [15:0] full_code;

assign full_code = {token[15:4], Code};

always_ff @(posedge CLK or posedge RST) begin
    if (RST) begin
        state <= S0;
        token <= 16'h0000;
        Unlock <= 1'b0;
        Err <= 1'b0;
    end else begin
        case (state)
            S0: begin
                token[15:12] <= Code;
                Unlock <= 1'b0;
                Err <= 1'b0;
                state <= S1;
            end
            S1: begin
                token[11:8] <= Code;
                state <= S2;
            end
            S2: begin
                token[7:4] <= Code;
                state <= S3;
            end
            S3: begin
                token[3:0] <= Code;
                if (full_code == password) begin
                    Unlock <= 1'b1;
                    Err <= 1'b0;
                end else begin
                    Unlock <= 1'b0;
                    Err <= 1'b1;
                end
            end
        end
    end
end
```



```

        state <= S4;
    end
    S4: begin
        state <= S4;
    end
    default: begin
        state <= S0;
    end
endcase
end
end

```

10. 模块实例化

10.1 为什么要拆模块

如果一个模块太大，你会乱。实验题里常拆：

```

top
  decoder
  lfsr
  tick_generator
  main_logic

```

10.2 实例化语法

假设有模块：

```

module decoder (
    input  logic [3:0] sw,
    output logic [6:0] seg
);
    ...
endmodule

```

在另一个模块里使用：

```

decoder u_low (
    .sw(low_digit),
    .seg(low_seg)
);

```

`u_low` 是实例名，随便起，但要唯一。

带参数模块：

```
module Lock #(
    parameter [15:0] DEFAULT_PASSWORD = 16'h1234
)(
    input logic CLK
);
    ...
endmodule
```

实例化时改参数：

```
Lock #(
    .DEFAULT_PASSWORD(16'h5678)
) u_lock (
    .CLK(CLK)
);
```

11. 输出到 LED 和数码管

11.1 LED

LED 最简单：

```
assign led = condition;
```

多位 LED：

```
assign LED[7:0] = shown_random;
```

复位时灭：

```
assign LED = RST ? 8'd0 : shown_random;
```

或者在寄存器里复位清零。

11.2 带译码数码管

历年题常说“带译码数码管”，这种数码管模块已经帮你把 4 位数字译码成七段显示。你只输出 4 位数字：

```
output logic [3:0] score_display
```

分数加一：

```
always_ff @(posedge CLK or posedge RST) begin
    if (RST)
        score <= 4'd0;
    else if (round_end && success) begin
        if (score == 4'd9)
            score <= 4'd0;
        else
            score <= score + 4'd1;
    end
end

assign score_display = score;
```

11.3 裸七段数码管

如果你的开发板要求 `seg[6:0]`，则需要 `decoder`。

```
decoder u_score (
    .sw(score),
    .seg(score_seg)
);
```

11.4 端口和管脚绑定

SV 代码里的端口：

```
output logic [3:0] Speed
```

约束文件里：

```
set_property -dict {PACKAGE_PIN M21 IOSTANDARD LVCMOS33} [get_ports Speed[3]]
set_property -dict {PACKAGE_PIN N20 IOSTANDARD LVCMOS33} [get_ports Speed[2]]
set_property -dict {PACKAGE_PIN N22 IOSTANDARD LVCMOS33} [get_ports Speed[1]]
set_property -dict {PACKAGE_PIN P21 IOSTANDARD LVCMOS33} [get_ports Speed[0]]
```

位序一定要核对。数码管显示不对时，第一怀疑就是位序接反。

12. 历年考试题的通用解题流程

拿到题后，不要急着写代码。按这个流程拆。

12.1 第一步：列端口

把题目里所有外部输入输出列出来：

输入：
CLK
RST
模式拨码
按键
开关

输出：
LED
数码管数字
报警灯

写成模块：

```
module exam_top (  
    input logic      CLK,  
    input logic      RST,  
    input logic      mode,  
    input logic      key,  
    input logic [3:0] sw,  
    output logic [7:0] LED,  
    output logic [3:0] D0,  
    output logic [3:0] D1  
);
```

12.2 第二步：找“需要记住”的量

题目里凡是“保存、上一次、当前显示、分数、速度、档位、锁定、次数、状态”，都要寄存器。

```
logic [7:0] shown_random;  
logic [7:0] saved_random;  
logic [3:0] score;  
logic [3:0] speed;  
logic [3:0] gear;  
logic [1:0] press_count;  
logic      locked;
```

12.3 第三步：找更新时间

什么时候更新？

```
每 1 秒 -> tick_1s  
每 2 秒 -> tick_2s  
每 3 秒 -> tick_3s
```

```
每 0.2 秒 -> tick_02s  
每次按键上升沿 -> key_rise  
每次手动时钟按下 -> posedge CLK  
复位 -> RST
```

12.4 第四步：写组合判断

把题目判定翻译成表达式。

```
assign dense = (ones_count >= 4'd4);  
assign win = (d1 + d2 + d3 >= 4'd10);  
assign hit_success = (hammer == mole) && confirm_pressed;  
assign gear = (speed >> 1) + 4'd1;
```

12.5 第五步：写时序更新

所有寄存器只在 `always_ff` 里更新。

```
always_ff @(posedge CLK or posedge RST) begin  
    if (RST) begin  
        // 清零或恢复初始值  
    end else begin  
        // 根据 tick、key_rise、mode 更新  
    end  
end
```

13. 题型模板一：随机显示类

对应题目：

1. 2024 打地鼠
2. 2024 老虎
3. 2025 序列检测器
4. 2025 电子骰子

核心结构：

```
LFSR 每个 CLK 都跑  
分频器产生 tick  
tick 到来时，把 random_out 锁存到 shown_random  
输出显示 shown_random 的某些位  
复位时显示 0
```

模板：

```

module random_display (
    input logic      CLK,
    input logic      RST,
    output logic [7:0] LED
);

    logic [7:0] random_out;
    logic [7:0] shown_random;
    logic [19:0] div_cnt;
    logic tick_1s;

    LFSR_random_generator u_lfsr (
        .clk(CLK),
        .reset(RST),
        .random_out(random_out)
    );

    always_ff @(posedge CLK or posedge RST) begin
        if (RST) begin
            div_cnt <= 20'd0;
            tick_1s <= 1'b0;
        end else begin
            if (div_cnt == 20'd999_999) begin
                div_cnt <= 20'd0;
                tick_1s <= 1'b1;
            end else begin
                div_cnt <= div_cnt + 20'd1;
                tick_1s <= 1'b0;
            end
        end
    end

    always_ff @(posedge CLK or posedge RST) begin
        if (RST)
            shown_random <= 8'd0;
        else if (tick_1s)
            shown_random <= random_out;
    end

    assign LED = shown_random;

endmodule

```

如果要求 2 秒，改分频终值。

如果要求暂停，给锁存加条件：

```

else if (tick_1s && !pause)
    shown_random <= random_out;

```

14. 题型模板二：分段数码管随机

老虎机：

```
random_out[5:4] -> 数码管 1
random_out[3:2] -> 数码管 2
random_out[1:0] -> 数码管 3
每 0.2 秒刷新
开关为 1 时对应数码管停止
```

核心写法：

```
logic [1:0] n0, n1, n2;
logic [7:0] random_out;

always_ff @(posedge CLK or posedge RST) begin
    if (RST) begin
        n0 <= 2'd0;
        n1 <= 2'd0;
        n2 <= 2'd0;
    end else if (tick_02s) begin
        if (!stop0)
            n0 <= random_out[1:0];
        if (!stop1)
            n1 <= random_out[3:2];
        if (!stop2)
            n2 <= random_out[5:4];
    end
end

assign D0 = {2'b00, n0};
assign D1 = {2'b00, n1};
assign D2 = {2'b00, n2};
```

如果要求一个按键按顺序停止右、中、左：

```
logic [1:0] lock_step;

always_ff @(posedge CLK or posedge RST) begin
    if (RST)
        lock_step <= 2'd0;
    else if (key_rise && lock_step < 2'd3)
        lock_step <= lock_step + 2'd1;
end

assign stop_right = (lock_step >= 2'd1);
```

```
assign stop_middle = (lock_step >= 2'd2);
assign stop_left   = (lock_step >= 2'd3);
```

15. 题型模板三：分数/成功判定

打地鼠：

地鼠图案每轮刷新
开关表示锤子
每轮结束时判断是否成功
成功分数 +1

核心：

```
logic [3:0] mole;
logic [3:0] hammer;
logic [3:0] score;
logic success;

assign success = (|(mole & hammer)) && !(|(hammer & ~mole));

always_ff @(posedge CLK or posedge RST) begin
    if (RST)
        score <= 4'd0;
    else if (round_end && success) begin
        if (score == 4'd9)
            score <= 4'd0;
        else
            score <= score + 4'd1;
    end
end
```

如果要求“必须打中所有地鼠且没打空”：

```
assign success = (hammer == mole) && (mole != 4'b0000) && confirm_seen;
```

如果确认按钮只要本轮内按过一次即可，需要锁存：

```
logic confirm_seen;

always_ff @(posedge CLK or posedge RST) begin
    if (RST)
        confirm_seen <= 1'b0;
    else if (round_end)
```



```
        confirm_seen <= 1'b0;
    else if (confirm_rise)
        confirm_seen <= 1'b1;
end
```

16. 题型模板四：汽车模拟驾驶

题意核心：

速度 0~9
档位 1~5
自动档：速度增加时自动根据速度更新档位
手动档：按手动换档开关档位增加，速度不能超过当前档位范围
刹车：每秒速度 -1，档位也要符合速度范围
复位：速度 0，档位 1

建议内部只保存 **speed**，档位用组合逻辑从速度算出。这样最不容易错。

```
logic [3:0] speed;
logic [3:0] gear;

always_comb begin
    if (speed <= 4'd1)
        gear = 4'd1;
    else if (speed <= 4'd3)
        gear = 4'd2;
    else if (speed <= 4'd5)
        gear = 4'd3;
    else if (speed <= 4'd7)
        gear = 4'd4;
    else
        gear = 4'd5;
end
```

自动加速/刹车：

```
always_ff @(posedge CLK or posedge RST) begin
    if (RST) begin
        speed <= 4'd0;
    end else if (tick_1s) begin
        if (brake) begin
            if (speed > 4'd0)
                speed <= speed - 4'd1;
        end else if (auto_mode) begin
            if (speed < 4'd9)
                speed <= speed + 4'd1;
        end
    end
end
```

```

        end
    end
end

```

手动档题目如果要求“手动换档时档位加一，速度不能超过该档位上限”，你可以换一种设计：保存 `manual_gear`，再限制速度。

```

logic [3:0] manual_gear;
logic [3:0] max_speed;

assign max_speed = manual_gear * 4'd2 - 4'd1;

always_ff @(posedge CLK or posedge RST) begin
    if (RST)
        manual_gear <= 4'd1;
    else if (!auto_mode && shift_rise) begin
        if (manual_gear < 4'd5)
            manual_gear <= manual_gear + 4'd1;
    end
end
end

```

但如果考试时间紧，优先保证基本功能和复位正确。进阶功能再逐步加。

17. 题型模板五：序列检测器

2025 题：

LFSR 产生 8 位随机序列
 每 2 秒 LED 更新
 统计 1 的数量
 >=4 显示 1，否则显示 0
 保存上一个 >=4 的序列
 暂停后按键切换查看当前序列和保存序列

核心代码：

```

logic [7:0] current_seq;
logic [7:0] saved_dense_seq;
logic [7:0] display_seq;
logic [3:0] ones_count;
logic dense;
logic show_saved;

assign ones_count = current_seq[0] + current_seq[1] + current_seq[2] +
    current_seq[3] +
        current_seq[4] + current_seq[5] + current_seq[6] +

```

```

current_seq[7];

assign dense = (ones_count >= 4'd4);

always_ff @(posedge CLK or posedge RST) begin
    if (RST) begin
        current_seq <= 8'd0;
        saved_dense_seq <= 8'd0;
    end else if (tick_2s && !pause_mode) begin
        current_seq <= random_out;
        if (dense)
            saved_dense_seq <= current_seq;
    end
end

```

上面有一个细节：`dense` 是根据旧的 `current_seq` 算的。如果你想保存“即将显示的新 `random_out`”，应该这样写：

```

logic [3:0] random_ones;
logic random_dense;

assign random_ones = random_out[0] + random_out[1] + random_out[2] + random_out[3]
+
                    random_out[4] + random_out[5] + random_out[6] +
random_out[7];
assign random_dense = (random_ones >= 4'd4);

always_ff @(posedge CLK or posedge RST) begin
    if (RST) begin
        current_seq <= 8'd0;
        saved_dense_seq <= 8'd0;
    end else if (tick_2s && !pause_mode) begin
        current_seq <= random_out;
        if (random_dense)
            saved_dense_seq <= random_out;
    end
end

```

暂停查看：

```

always_ff @(posedge CLK or posedge RST) begin
    if (RST)
        show_saved <= 1'b0;
    else if (pause_mode && key_rise)
        show_saved <= ~show_saved;
    else if (!pause_mode)
        show_saved <= 1'b0;
end

```

```
assign display_seq = (pause_mode && show_saved) ? saved_dense_seq : current_seq;
assign LED = RST ? 8'd0 : display_seq;
assign digit = dense ? 4'd1 : 4'd0;
```

18. Testbench 入门

18.1 组合逻辑 testbench

测四位加法器：

```
module adder_tb;

    logic [3:0] A;
    logic [3:0] B;
    logic      Cin;
    logic [3:0] F;
    logic      Cout;

    int i, j, k;
    int expected;

    adder dut (
        .A(A),
        .B(B),
        .Cin(Cin),
        .F(F),
        .Cout(Cout)
    );

    initial begin
        for (i = 0; i < 16; i++) begin
            for (j = 0; j < 16; j++) begin
                for (k = 0; k < 2; k++) begin
                    A = i[3:0];
                    B = j[3:0];
                    Cin = k[0];
                    #1;

                    expected = i + j + k;

                    if ({Cout, F} != expected[4:0]) begin
                        $display("ERROR A=%0d B=%0d Cin=%0d got=%0d expected=%0d",
                            i, j, k, {Cout, F}, expected);
                    end
                end
            end
        end

        $display("finish");
        $finish;
    end
endmodule
```

```
end

endmodule
```

18.2 时序逻辑 testbench

产生时钟：

```
initial CLK = 1'b0;
always #500 CLK = ~CLK; // 周期 1000 ns, 即 1 MHz
```

复位：

```
initial begin
    RST = 1'b1;
    repeat (2) @(posedge CLK);
    RST = 1'b0;
end
```

等待若干时钟：

```
repeat (10) @(posedge CLK);
```

等待一个 tick：

```
@(posedge dut.tick_1s);
```

手动按键：

```
task automatic press_key;
    begin
        key = 1'b0;
        @(posedge CLK);
        key = 1'b1;
        @(posedge CLK);
        key = 1'b0;
        @(posedge CLK);
    end
endtask
```

18.3 testbench 不综合

这些只能在仿真里用：

```
initial
#10
$display
$finish
task
int
```

设计文件里别用 `#10` 延时，硬件不会综合出“等 10ns 再执行”的普通逻辑。设计里要靠计数器。

19. 常见错误清单

19.1 组合逻辑漏赋值

错：

```
always_comb begin
    if (a)
        y = 1'b1;
end
```

对：

```
always_comb begin
    y = 1'b0;
    if (a)
        y = 1'b1;
end
```

19.2 一个信号多个驱动

错：

```
assign led = a;

always_ff @(posedge CLK) begin
    led <= b;
end
```

`led` 同时被 `assign` 和 `always_ff` 驱动。

对：只保留一种驱动。

19.3 时序逻辑里用阻塞赋值

不推荐：

```
always_ff @(posedge CLK) begin
    count = count + 1;
end
```

推荐：

```
always_ff @(posedge CLK) begin
    count <= count + 1;
end
```

19.4 忘记复位

考试题几乎都要求复位。所有重要寄存器都要在 `if (RST)` 里给初值。

19.5 位宽不够

错：

```
logic [3:0] sum;
assign sum = A + B; // 可能溢出
```

对：

```
logic [4:0] sum;
assign sum = A + B;
```

19.6 直接显示高速随机数

错：

```
assign LED = random_out;
```

如果 LFSR 用 1 MHz 时钟，LED 会高速变化。

对：

```
always_ff @(posedge CLK or posedge RST) begin
    if (RST)
```

```

        shown <= 8'd0;
    else if (tick_1s)
        shown <= random_out;
end

assign LED = shown;

```

19.7 没理解非阻塞旧值

错：

```

always_ff @(posedge CLK) begin
    d3 <= r3;
    led <= ((d1 + d2 + d3) >= 10); // 这里 d3 还是旧值
end

```

对：

```

always_ff @(posedge CLK) begin
    d3 <= r3;
    led <= ((d1 + d2 + r3) >= 10);
end

```

19.8 手动按键和主时钟混乱

如果 **CLK** 本身就是手动按键：

```

always_ff @(posedge CLK or posedge RST)

```

每按一次就触发一次。

如果 **CLK** 是 1 MHz，按键是另一个输入 **key**：

```

// 用 key_rise 检测按键上升沿

```

不要把普通按键直接当作另一个时钟乱接多个 always 块，除非题目明确要求。

20. 从题面到代码的完整例子：电子骰子简化版

需求：

LFSR 生成随机数

三组骰子每 1 秒刷新

按键第一次锁 d1, 第二次锁 d2, 第三次锁 d3 并判断总和 ≥ 10 点亮 LED

复位清零

输出到三个带译码数码管

参考代码:

```
module dice_exam (
    input logic      CLK,
    input logic      RST,
    input logic      key,
    output logic [3:0] D1,
    output logic [3:0] D2,
    output logic [3:0] D3,
    output logic      LED
);

    logic [7:0] random_out;
    logic [31:0] div_cnt;
    logic tick_1s;

    logic key_d;
    logic key_rise;

    logic [3:0] r1, r2, r3;
    logic [3:0] d1, d2, d3;

    typedef enum logic [1:0] {
        ROLL_ALL = 2'd0,
        LOCK_1   = 2'd1,
        LOCK_2   = 2'd2,
        LOCK_3   = 2'd3
    } state_t;

    state_t state;

    LFSR_random_generator u_lfsr (
        .clk(CLK),
        .reset(RST),
        .random_out(random_out)
    );

    assign r1 = {2'b00, random_out[7:6]};
    assign r2 = {1'b0,  random_out[5:3]};
    assign r3 = {1'b0,  random_out[2:0]};

    always_ff @(posedge CLK or posedge RST) begin
        if (RST) begin
            div_cnt <= 32'd0;

```

```

        tick_1s <= 1'b0;
    end else begin
        if (div_cnt == 32'd999_999) begin
            div_cnt <= 32'd0;
            tick_1s <= 1'b1;
        end else begin
            div_cnt <= div_cnt + 32'd1;
            tick_1s <= 1'b0;
        end
    end
end

always_ff @(posedge CLK or posedge RST) begin
    if (RST)
        key_d <= 1'b0;
    else
        key_d <= key;
end

assign key_rise = key & ~key_d;

always_ff @(posedge CLK or posedge RST) begin
    if (RST) begin
        state <= ROLL_ALL;
        d1 <= 4'd0;
        d2 <= 4'd0;
        d3 <= 4'd0;
        LED <= 1'b0;
    end else begin
        if (tick_1s) begin
            if (state == ROLL_ALL)
                d1 <= r1;
            if (state == ROLL_ALL || state == LOCK_1)
                d2 <= r2;
            if (state == ROLL_ALL || state == LOCK_1 || state == LOCK_2)
                d3 <= r3;
        end

        if (key_rise) begin
            case (state)
                ROLL_ALL: begin
                    d1 <= r1;
                    state <= LOCK_1;
                end
                LOCK_1: begin
                    d2 <= r2;
                    state <= LOCK_2;
                end
                LOCK_2: begin
                    d3 <= r3;
                    LED <= ((d1 + d2 + r3) >= 4'd10);
                    state <= LOCK_3;
                end
                default: begin

```

```
                state <= LOCK_3;
            end
        endcase
    end
end
end

    assign D1 = d1;
    assign D2 = d2;
    assign D3 = d3;

endmodule
```

这段代码包含考试最核心的几件事：

1. LFSR 实例化。
2. 1 秒 tick。
3. 按键上升沿。
4. 分步锁定状态机。
5. 非阻塞赋值旧值问题。
6. 带译码数码管输出 4 位数字。

21. 实操训练路线

只看讲义不够，必须敲。按这个顺序练，能最快达到考试可写水平。

第 1 天：语法和组合逻辑

练习：

1. 写一个 2 输入与门、或门、异或门。
2. 写一个 2 选 1 选择器。
3. 写一个 4 位比较器，大于等于 10 输出 1。
4. 写一个七段译码器。
5. 写一个 4 位加法器，输出 5 位结果。

目标：

看到 `input/output/logic/[3:0]/assign/always_comb/case` 不再陌生

第 2 天：时序逻辑

练习：

1. 写一个 0~9 循环计数器。
2. 写一个带 **RUN** 暂停的计数器。
3. 写一个 1 MHz 到 1 秒的分频器。
4. 写一个 00~59 秒表。

目标：

会用 `always_ff`、复位、非阻塞赋值、`tick`

第 3 天：随机数和显示

练习：

1. 实例化 LFSR。
2. 每 1 秒把 LFSR 输出锁存到 LED。
3. 把 `random_out[5:4]`、`[3:2]`、`[1:0]` 输出到三个数码管。
4. 加三个 stop 开关，分别停止三个数码管刷新。

目标：

能做老虎机/随机显示类基本题

第 4 天：按键和状态机

练习：

1. 写按键上升沿检测。
2. 按一次锁一个数码管，三次全部锁住。
3. 写密码锁四位输入 FSM。
4. 写分数计数器。

目标：

能做骰子、密码锁、打地鼠进阶题

第 5 天：整题训练

从历年题里任选两题，限时做：

1. 先做要求 1。
2. 再做要求 2。
3. 最后做要求 3。

每个要求都先仿真再继续。

考试时不要一口气写全部功能。每完成一个小要求就确保它逻辑自治。

22. 考试时的代码骨架

你可以先默写这个骨架：

```
module exam_top (
    input logic      CLK,
    input logic      RST,
    input logic      key,
    input logic      mode,
    input logic [3:0] sw,
    output logic [7:0] LED,
    output logic [3:0] D0,
    output logic [3:0] D1,
    output logic [3:0] D2
);

// 1. 内部信号
logic [7:0] random_out;
logic [31:0] div_cnt;
logic tick;

logic key_d;
logic key_rise;

// 2. LFSR
LFSR_random_generator u_lfsr (
    .clk(CLK),
    .reset(RST),
    .random_out(random_out)
);

// 3. 分频
always_ff @(posedge CLK or posedge RST) begin
    if (RST) begin
        div_cnt <= 32'd0;
        tick <= 1'b0;
    end else begin
        if (div_cnt == 32'd999_999) begin
            div_cnt <= 32'd0;
            tick <= 1'b1;
        end else begin
            div_cnt <= div_cnt + 32'd1;
            tick <= 1'b0;
        end
    end
end

// 4. 按键上升沿
always_ff @(posedge CLK or posedge RST) begin
    if (RST)
        key_d <= 1'b0;
    else
        key_d <= key;
end

assign key_rise = key & ~key_d;
```

```
// 5. 主时序逻辑
always_ff @(posedge CLK or posedge RST) begin
    if (RST) begin
        // 所有寄存器复位
    end else begin
        if (tick) begin
            // 周期刷新
        end

        if (key_rise) begin
            // 按键事件
        end
    end
end

// 6. 组合输出
always_comb begin
    // 根据状态决定输出
end

endmodule
```

23. 最后背熟的十句话

1. `module` 是硬件模块，端口名要和 `.xdc` 对上。
2. `[3:0]` 是 4 位，`[3]` 高位，`[0]` 低位。
3. 数字写成 `4'd9`、`8'hFF`、`20'd999_999`。
4. 简单组合逻辑用 `assign`。
5. 复杂组合逻辑用 `always_comb`，必须每条路径都赋值。
6. 要记住历史就用 `always_ff @(posedge CLK or posedge RST)`。
7. `always_comb` 用 `=`，`always_ff` 用 `<=`。
8. 分频本质是计数器，到终值产生一个 tick。
9. 按键上升沿检测要保存上一拍：`key_rise = key & ~key_d`。
10. 考试题先拆端口、寄存器、更新时间、组合判断，再写代码。

只要你能熟练写出：译码器、加法器、分频器、计数器、边沿检测、LFSR 锁存、简单状态机，本课程实验考试的大多数题就都能拆开做。